



GLOBE & TAIPEI ECONOMIC AND
CULTURAL OFFICE IN SAN
FRANCISCO (TECO)
PRESENT

A CHIP ODYSSEY

FILM SCREENING & PANEL DISCUSSION

Friday, April 24 | 3:30 PM–7:30 PM | Jarvis Auditorium, Grimes Engineering Center



Chenming Hu
TSMC Distinguished
Professor Emeritus



Clark Nguyen
Engineering Executive
Associate Dean

EVENT HIGHLIGHTS

3:30 PM – Doors Open

4:30 PM – Film Screening

6:20 PM – Panel Q&A

6:50 PM – Reception



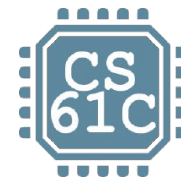
Scan to see the film
trailer & more event
details!





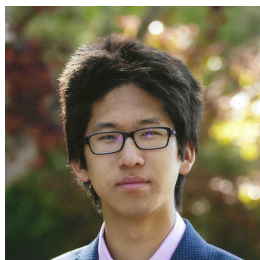
CS 61C Industry & Research Panel

May 5 (Tue), 2-3pm, Wozniak Lounge; bit.ly/61c-panel
Complementary pizza will be provided



Nicolas Rakela

- BS EECS'26 - UCB
- MS EECS'27 - UCB
- SLICE/BWRC Lab
- EECS 16A, EECS 16B, EECS 151, Tapeout TA
- Apple Intern ('25, '26)
- Apple MS Scholar



Jim Fang

- BS EECS'27 - UCB
- SLICE/BWRC Lab
- Former CS 61C TA
- Tapeout Head TA
- IEEE Chair
- Google Student Researcher ('25-'26)



Professor Sagar Karandikar

- BS EECS'15 - UCB
- PhD EECS'24 - UCB
- SLICE/BWRC Lab
- Former 6x CS 61C TA (4x as Head TA) and Lecturer
- Google Student/Faculty Researcher ('20-'25)



- Professor Chris Fletcher**
- BS EECS'10 - UCB
 - PhD EECS'16 - MIT
 - SLICE/BWRC Lab
 - Former UIUC Professor
 - NVIDIA Architecture Research Group ('16-'17)



Andre Green

- BS EE'23 - Penn
- MS EECS'25 - UCB
- PhD EECS'28 - UCB
- SLICE/BWRC Lab
- Former CS61C TA
- IBM Intern ('22-'23)
- NVIDIA Architecture Research Group ('25-)



Kelly Tou

- BS EECS'27 - UCB
- Former SWARM Lab
- SLICE/BWRC Lab
- EECS 151 TA
- 2x Apple NSI Design Winner
- Apple Intern ('25, '26)



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

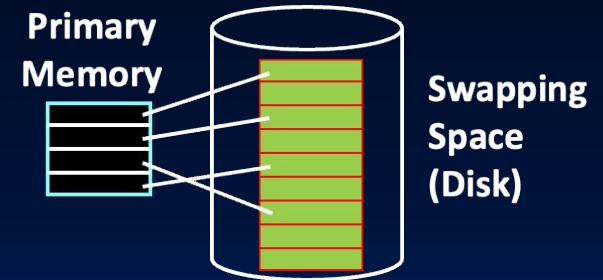
Virtual Memory II



Modern Virtual Memory Systems

[review]

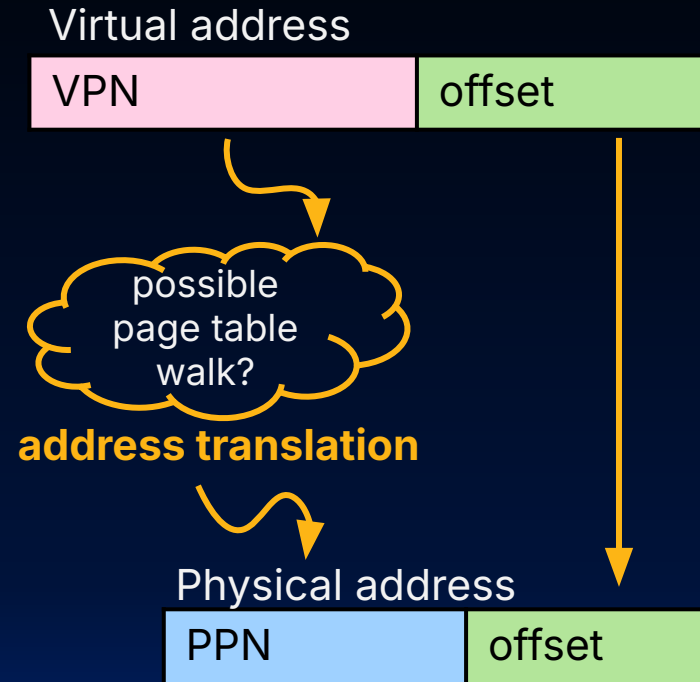
- Modern Virtual Memory Systems use address translation to provide the illusion of a large, **private**, and **uniform storage**.
 - Privacy means Protection:**
 - Several users/processes, each with their own private address space.
 - Uniform storage means Demand Paging:**
 - The ability to run programs larger than main memory (DRAM).
 - Hides difference in machine configurations.
- Price: Address translation (via **page tables**) on each memory reference.



Avoid Page Table Walks

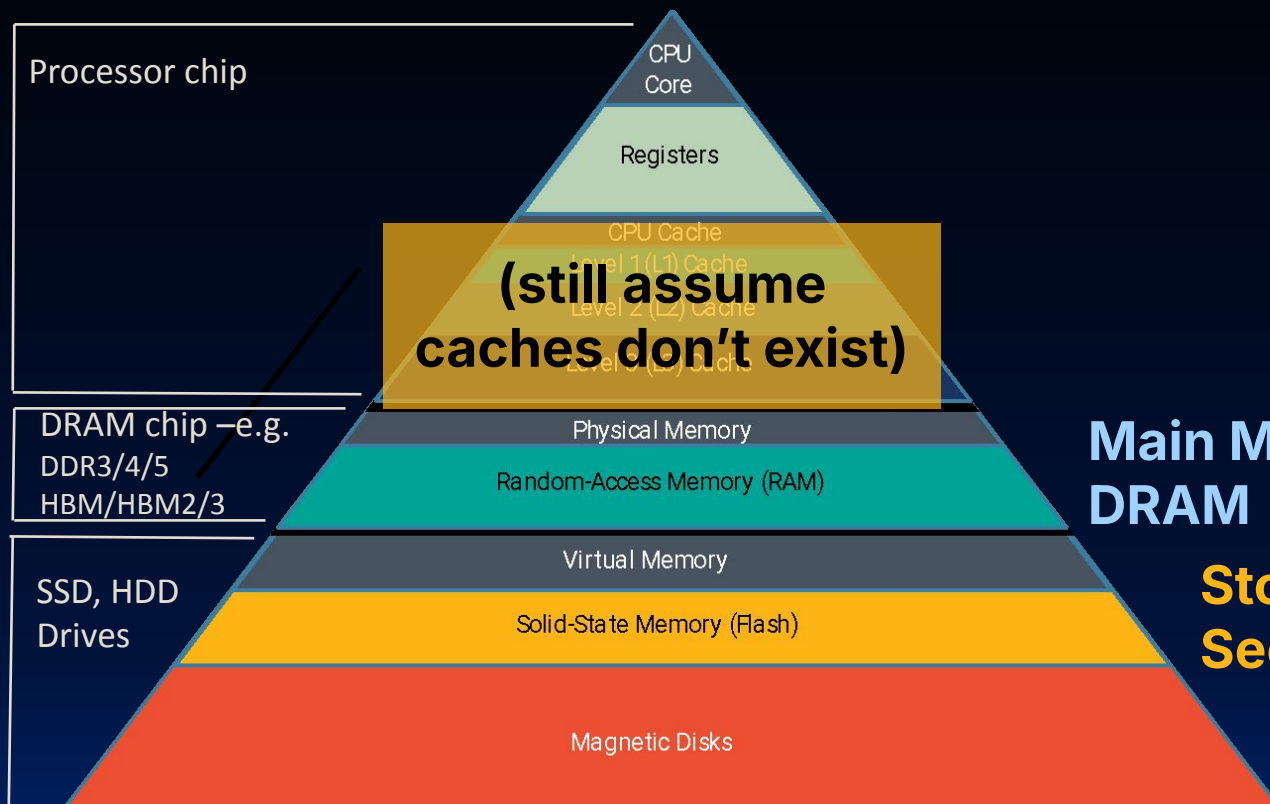
[review]

- Good Virtual Memory design should be fast (~1 clock cycle) and space efficient.
 - However, every instruction/data access needs **address translation**.
- A **page table walk** is the process of accessing the page table in memory.
 - If page tables are in memory, then we will need **TWO memory accesses** per instruction/data access!
- Solution: Cache some address translations in the **Translation Lookaside Buffer (TLB)**. (today)



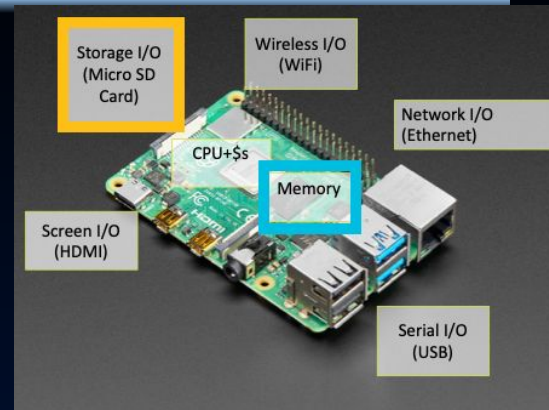
Main Memory and Secondary Memory

[review]



**Main Memory /
DRAM**

**Storage / "Disk" /
Secondary Memory**





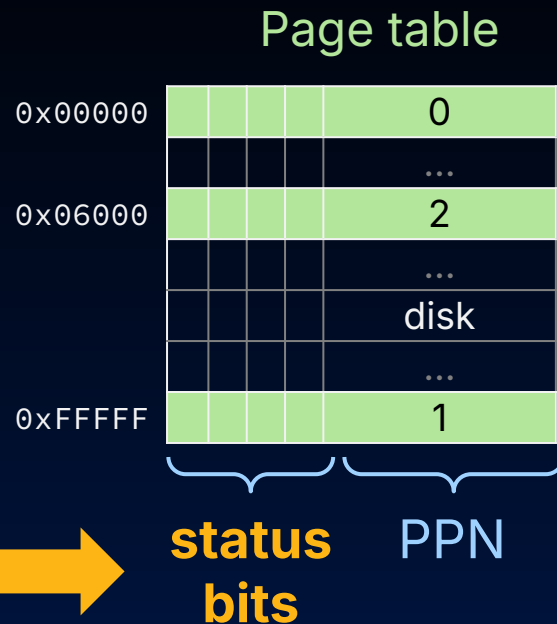
Page Table Details II

- Page Table Details II
- The Translation Lookaside Buffer (TLB)
- Address Translation: Best/Worst Case
- Caches vs. Virtual Memory

What Do Page Tables Look Like? Status Bits

[review]

- One page table per process.
 - One entry per virtual page number.
 - Entry has **physical page number** (if in memory)
 - Entry has **status bits**: valid, dirty, write protection (discussed later now)
- We now discuss three status bits:
 - Valid Bit
 - Dirty bit
 - Write protection bit



Why track these bits?





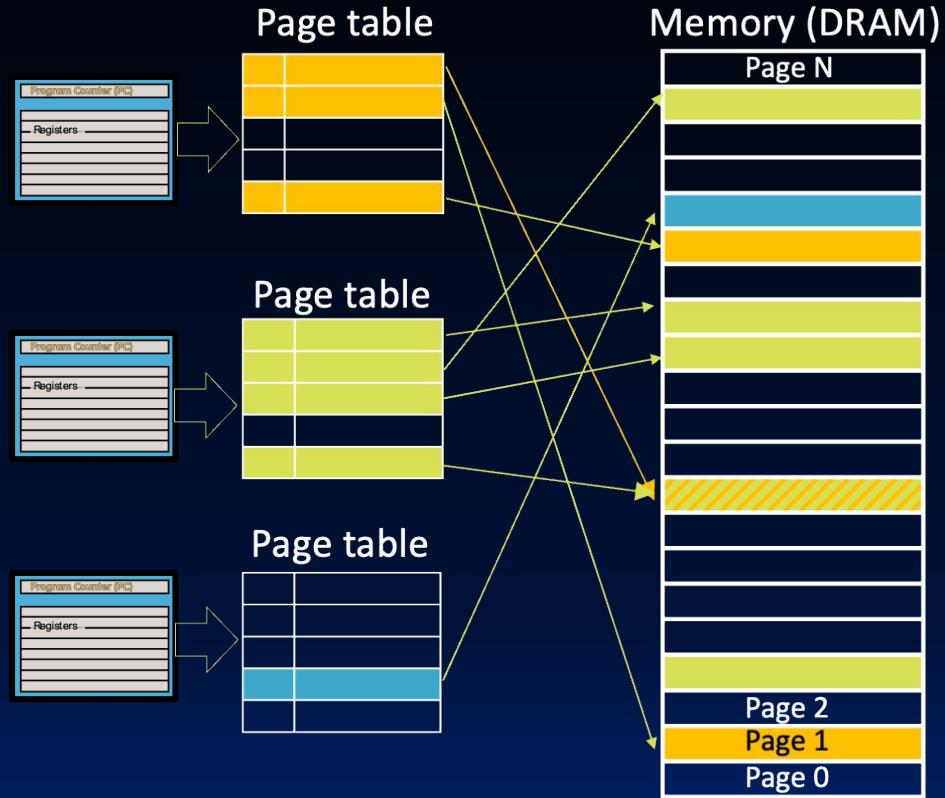
Status Bits

1. Page table entries track a **valid bit** to indicate if the page is in memory (DRAM) or only on disk.
 - a. On each memory access, first check if page table entry is "valid."
 - b. If not valid, page fault, load page into memory, then do access
2. All virtual memory systems use **write-back**.
 - a. Disk accesses take way too long!
 - b. When a page gets replaced, if dirty bit on: Write outgoing page back to disk.
3. **Write protection bit** for page table entries shared b/t processes
 - a. (next slide)

More in
refslides

Write Protection (1/2): Per-process Page Tables

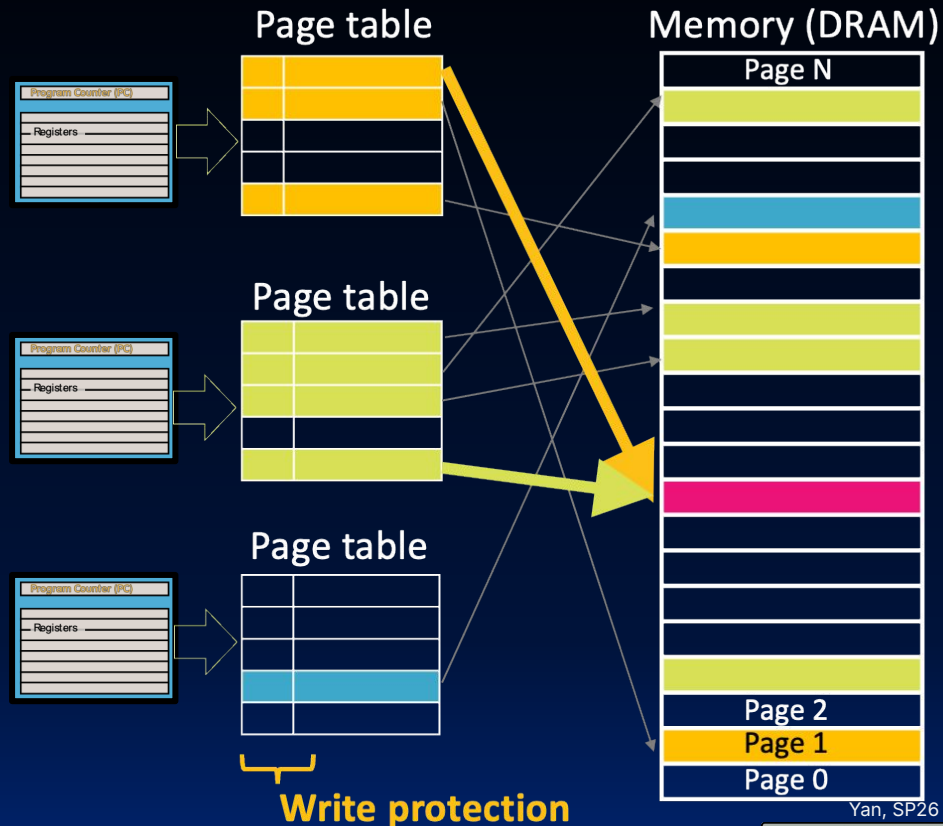
- Each process has a **dedicated page table**.
- Isolation**: Assign processes different pages in DRAM.
 - Prevents accessing other processors' memory
 - Page tables managed by OS
- OS keeps track of which process is active.



Yan, SP26

Write Protection (2/2): Status Bit for Safe Memory Sharing

- Multiple programs could share memory.
 - Just direct both programs to the **same physical page**.
 - Example: Programs might need the same library code, e.g. `stdlib.h`.
- Page table **write protection bit**: on := page is "protected."
 - Writing to a protected page triggers an OS exception (more later).
 - Example: program code, system data, etc.





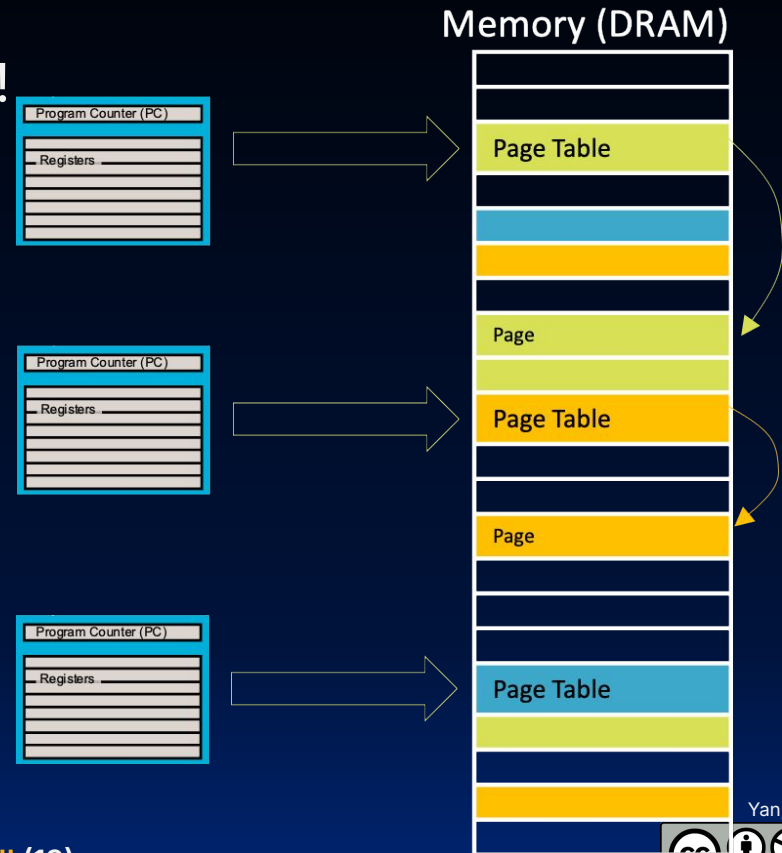
The Translation Lookaside Buffer (TLB)

- Page Table Details II
- The Translation Lookaside Buffer (TLB)
- Address Translation: Best/Worst Case
- Caches vs. Virtual Memory

Leverage caches for both memory accesses

- **Caveat:** Two (slow) memory accesses per lw/sw on cache miss!
 - a. **Address translation** with **Page table walk**: Read **page table** (stored in main memory)
 - b. Read **physical page**, also in main memory
- To minimize performance penalty:
 - a. Use a cache for frequently used page table entries: **TLB**
 - b. Transfer **cache blocks** (not words) between DRAM and processor cache

We know how to do (b).
Let's check out (a).



The TLB Caches Address Lookups

- The **translation lookaside buffer (TLB)** is a cache of VPN-PPN mappings.
- On TLB hit, avoid page table walk!

VPN	PPN
0x00004	0x60C25E6
0x00005	0x71DB139
0x00009	0x45099CD

The TLB

I just remember the last 3 VPN/PPN mappings I've seen.

The TLB is a cache—not for program data, but for **page table entries**. Speeds up **address translation**!

VPN	PPN
...	...
0x00004	0x60C25E6
0x00005	0x71DB139
0x00006	0xEC70DB7
0x00007	0xAB12BF4
0x00008	0x2158D55
0x00009	0x45099CD
...	...

I have every VPN/PPN mapping, from VPN 0x00000 to 0xFFFFF.

Process 1
Page Table



TLB Details (1/3): Design

- Technical details:
 - 32-128 entries
 - Fully associative (or 2-way set associative)
 - Replacement policy: FIFO, random
 - Tracks status bits of page table entry

VPN	PPN
0x00004	0x60C25E6
0x00005	0x71DB139
0x00009	0x45099CD

The TLB

NB: Instead of thinking about associativity, tag/index/offset, etc....easier to think:
"the last few VPN/PPN mappings I've seen."

TLB Details (2/3): TLB Reach

- Technical details:
 - 38-128 entries
 - Fully associative (or 2-way set associative)
 - Replacement policy: FIFO, random
 - Tracks status bits of page table entry
- **TLB Reach** tells us how many **virtual addresses** can get immediately translated by the TLB.
 - **TLB Reach = # TLB entries x Page size**

VPN	PPN
0x00004	0x60C25E6
0x00005	0x71DB139
0x00009	0x45099CD

The TLB

I support instant lookup
for up to $3 \times 2^{12} = 12288$
virtual addresses.

(assuming 4KiB pages,
i.e., 12-bit page offset)

TLB Details (3/3): TLB Flushing and Valid Entries

- Technical details:
 - 38-128 entries
 - Fully associative (or 2-way set associative)
 - Replacement policy: FIFO, random
 - Tracks status bits of page table entry
- TLB Reach** tells us how many **virtual addresses** can get immediately translated by the TLB.
 - TLB Reach = # TLB entries x Page size**

v	VPN	PPN
0	0x00001	0x60C25E6
0	0x00005	0x71DB139
0	0x00009	0x45099CD

The TLB

v	Block
0	
0	
0	

- There is just **one TLB per core**, but recall page tables are per process.
- When the OS performs a context switch to run a different program:
 - Easiest: OS **flushes** all/part of the TLB by invalidating its entries.
 - Harder: track which process corresponds to which entries

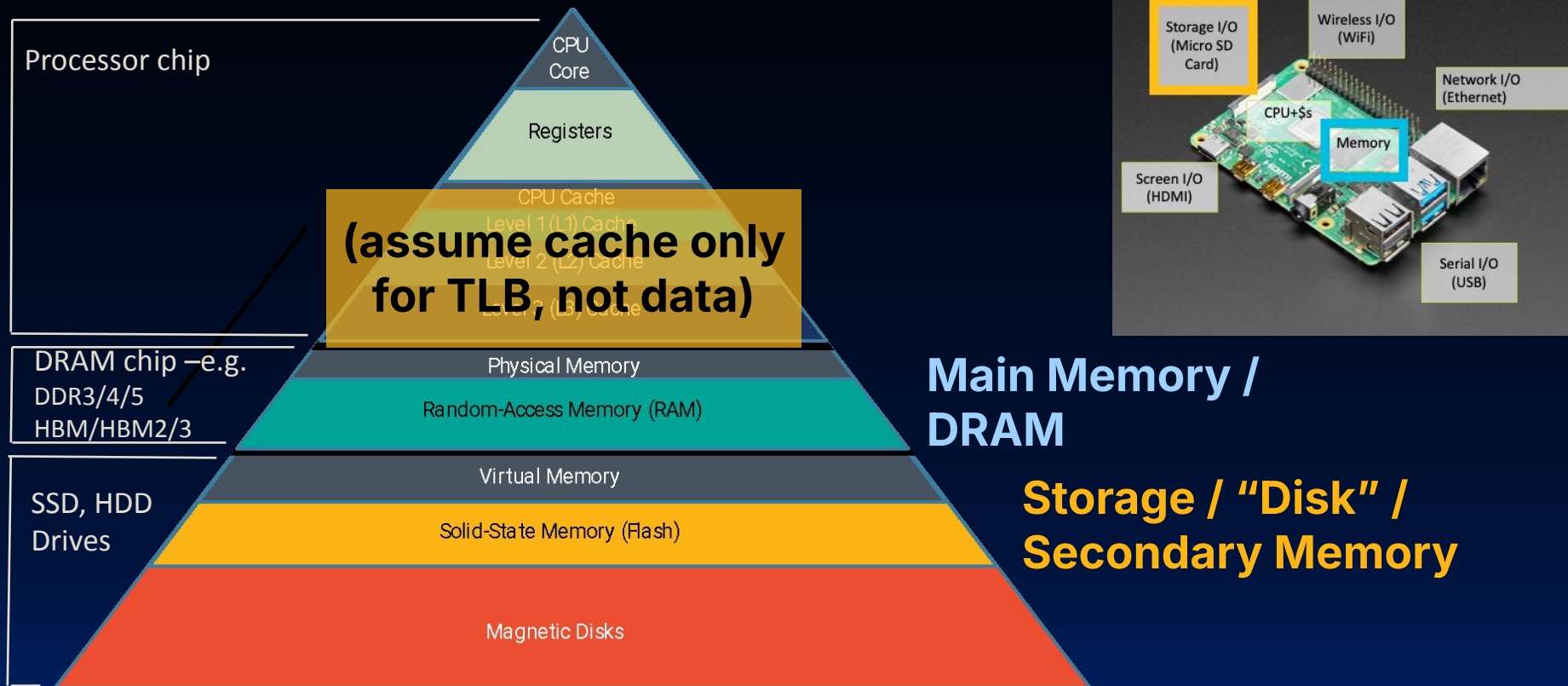


Address Translation: Best/Worst Case

- Page Table Details II
- The Translation Lookaside Buffer (TLB)
- Address Translation: Best/Worst Case
- Caches vs. Virtual Memory

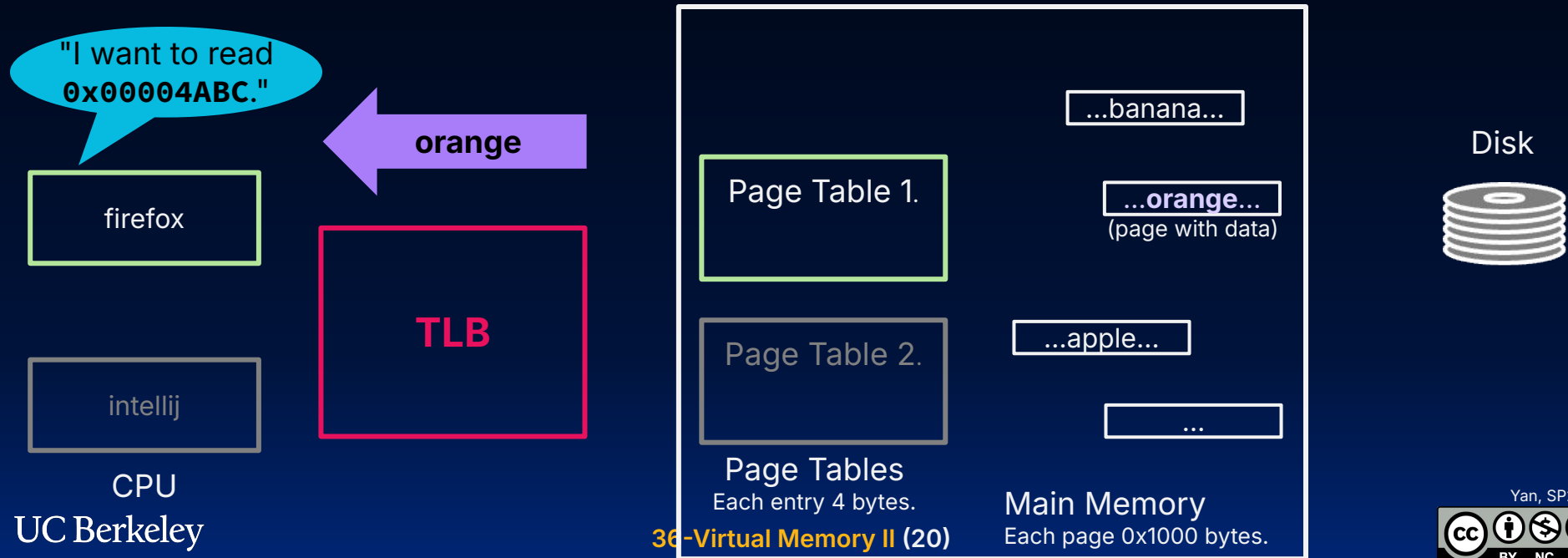
Address Translation: Keep it "simple"

[review]



Address Translation Example

- 3 different cases of **address translation** impact the **end-to-end latency** of a single memory access.
 - Memory access = address translation + data access





[Case 1, Best] TLB Hit

- Address Translation: Instant (~1 clock cycle)

"I want to read
0x00004ABC."

VPN 4 in TLB!



firefox

```
lw  s5 12(a2)
srli t2 t0 3
...
```

intellij

```
addi s2 x0 3
jal  label
...
```

CPU

UC Berkeley

VPN	PPN
0x00004	0x8C121D
0x00005	0x2158D55
0x00009	0x45099CD

TLB

VPN 3
VPN 4
VPN 5

...
0xAB12BF5
0x82C121D
0xD01A3F1
...

VPN 3
VPN 4
VPN 5

...
0xAB12BF4
disk
0x2158D55
...

Page Tables

Each entry 4 bytes.

36-Virtual Memory II (21)

...
banana
...
apple
potato
...
orange
...
carrot
...
Page Table
Page Table

...

0xD01A3F1000

...

0xAB12BF5000

0xAB12BF4000

...

0x82C121D000

...

0x2158D55000

...

0x120331D000

...

0x120331C000

Main Memory

Each page 0x1000 bytes.

Disk

...
...
...
beans
orange
...
...
...
...

Yan, SP26



[Case 2, Worse] TLB Miss + Page Table Walk

- Page table walk (~100 cycles): Go to main memory, read page table.

"I want to read
0x00004ABC."

VPN 4 NOT in TLB.



firefox

```
lw s5 12(a2)
srli t2 t0 3
...
```

intellij

```
addi s2 x0 3
jal label
...
```

CPU

UC Berkeley

VPN	PPN
0x00002	0x30219D
0x00005	0x2158D55
0x00009	0x45099CD

TLB

VPN 4 in
Page Table!



...
VPN 3 0xAB12BF5
VPN 4 0x82C121D
VPN 5 0xD01A3F1
...

...
VPN 3 0xAB12BF4
VPN 4 disk
VPN 5 0x2158D55
...

Page Tables

Each entry 4 bytes.

36-Virtual Memory II (22)

...	...
banana	0xD01A3F1000
...	...
apple	0xAB12BF5000
potato	0xAB12BF4000
...	...
orange	0x82C121D000
...	...
carrot	0x2158D55000
...	...
Page Table	0x120331D000
Page Table	0x120331C000

Main Memory

Each page 0x1000 bytes.

Disk

...
...
...
beans
orange
...
...
...
...

Yan, SP26

[Case 2, Worse] TLB Miss + Page Table Walk, cont.

- Page table walk (~100 cycles): Go to main memory, read page table.
- Update TLB** so we have this VPN/PPN mapping handy for next time.

"I want to read
0x00004ABC."

VPN 4 NOT in TLB.

Update TLB.

VPN	PPN
0x00004	0x8C121D
0x00005	0x2158D55
0x00009	0x45099CD

TLB

VPN 4 in
Page Table!



	...
VPN 3	0xAB12BF5
VPN 4	0x82C121D
VPN 5	0xD01A3F1
	...

	...
VPN 3	0xAB12BF4
VPN 4	disk
VPN 5	0x2158D55
	...

Page Tables

Each entry 4 bytes.

36-Virtual Memory II (23)

...	...
banana	0xD01A3F1000
...	...
apple	0xAB12BF5000
potato	0xAB12BF4000
...	...
orange	0x82C121D000
...	...
carrot	0x2158D55000
...	...
Page Table	0x120331D000
Page Table	0x120331C000

Main Memory

Each page 0x1000 bytes.

Disk

...
...
...
beans
orange
...
...
...

Yan, SP26

[Case 3, Worst] Page Fault

- Page table walk+page fault. Go to disk, load page (~1000s cycles)

"I want to read
0x00004ABC."

VPN 4 NOT in TLB.



firefox

```
lw    s5 12(a2)
srli  t2 t0 3
...
```

intellij

```
addi  s2 x0 3
jal   label
...
```

CPU

UC Berkeley

VPN	PPN
0x00002	0x30219D
0x00005	0x2158D55
0x00009	0x45099CD

TLB

VPN 4 has
no PPN in
Page Table.



...
VPN 3 0xAB12BF5
VPN 4 disk
VPN 5 0xD01A3F1
...

...
VPN 3 0xAB12BF4
VPN 4 disk
VPN 5 0x2158D55
...

Page Tables

Each entry 4 bytes.

36-Virtual Memory II (24)

...	...
banana	0xD01A3F1000
...	...
apple	0xAB12BF5000
potato	0xAB12BF4000
...	...
orange	0x82C121D000
...	...
carrot	0x2158D55000
...	...
Page Table	0x120331D000
Page Table	0x120331C000

Main Memory

Each page 0x1000 bytes.

Load page
from disk.

Disk

...
...
...
beans
orange
...
...
...

Yan, SP26

[Case 3, Worst] Page Fault, cont.

- Page table walk+page fault. Go to disk, load page (~1000s cycles)
- Update page table** with PPN of the newly-loaded page.
- Update TLB** so we have this VPN/PPN mapping handy for next time.

"I want to read
0x00004ABC."

VPN 4 NOT in TLB.

Update TLB.

VPN	PPN
0x00004	0x8C121D
0x00005	0x2158D55
0x00009	0x45099CD

TLB

VPN 4 has
no PPN in
Page Table.

VPN 3	0xAB12BF5
VPN 4	0x82C121D
VPN 5	0xD01A3F1

Update page table.

VPN 3	0xAB12BF4
VPN 4	disk
VPN 5	0x2158D55

Page Tables

Each entry 4 bytes.

36-Virtual Memory II (25)

...	...
banana	0xD01A3F1000
...	...
apple	0xAB12BF5000
potato	0xAB12BF4000
...	...
orange	0x82C121D000
...	...
carrot	0x2158D55000
...	...
Page Table	0x120331D000
Page Table	0x120331C000

Main Memory

Each page 0x1000 bytes.

**Load page
from disk.**

...
...
...
beans
orange
...
...
...

Yan, SP26

Summary: TLB Hits, Page Table Walk, Page Fault...

- Three cases for an address translation:

	TLB	Page Table Walk	Disk access
Best	Hit 	Not visited	Not visited
Worse	Miss 	Page Table Entry Valid 	Not visited
Worst	Miss 	Page Fault 	Load page 

Remember: The TLB caches **recent** page table entries. If the entry is valid in the TLB, it **must also** be in the page table, and the data must be in memory.

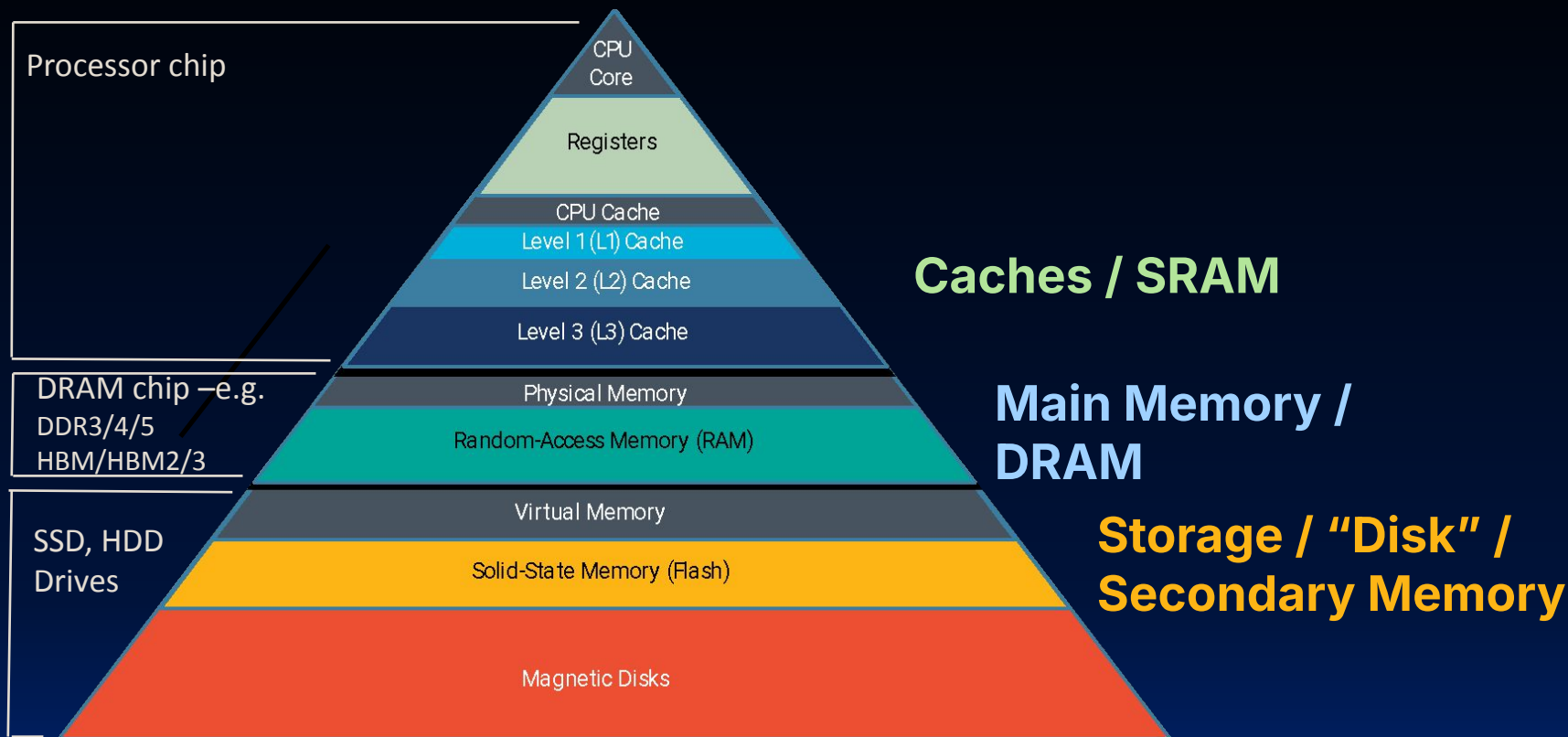


Caches vs. Virtual Memory

- Page Table Details II
- The Translation Lookaside Buffer (TLB)
- Address Translation: Best/Worst Case
- Caches vs. Virtual Memory



The Full Modern Memory Hierarchy





Caches vs. Virtual Memory

Caches

Virtual Memory

In memory
hierarchy

Caches $\leftrightarrow$ Memory

Memory $\leftrightarrow$ Disk

Memory unit

Miss

Associativity

Replacement
policy

Write policy

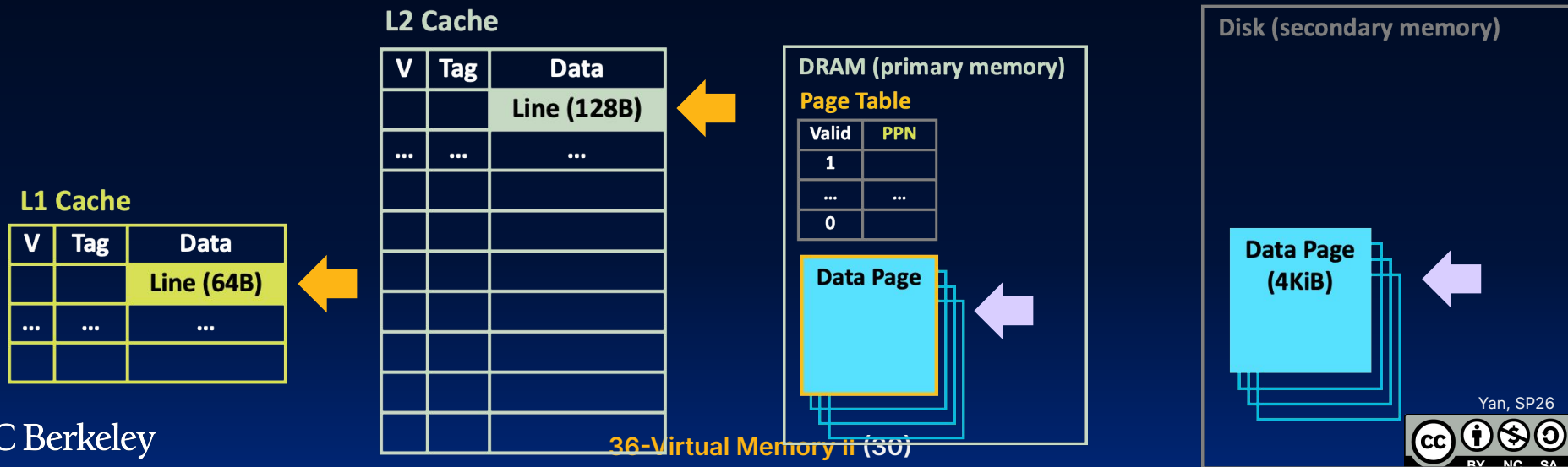
What choices
might we have
for VM? Why?



Yan, SP26

Terminology: Memory Units

- Blocks/lines, pages, words, bytes are all units in the memory hierarchy.
- Caches: **blocks** or **lines**
 - On modern systems, 64B or 128B
- Memory: **pages**
 - On modern systems, 4KiB



Caches Have Copies of Data

- **"Cache" Paradigm:** Data at each level is a quick-access copy of data at a lower level in the memory hierarchy.
 - Similarly, data pages in main memory are "cached" versions of pages on disk.

L1 Cache

V	Tag	Data
		☺
...	...	...

L2 Cache

V	Tag	Data
		☺
...	...	...

DRAM (primary memory)

Page Table

Valid	PPN
1	
...	...
0	

Data Page



Disk (secondary memory)

Data Page (4KiB)



Page Tables Do Not Have Copies of Data

- A Page Table translates addresses.
- Page tables facilitate **demand paging**.
 - Cache data pages in memory.
 - Access disk pages only when needed by the process.
 - Page Table keeps track of page status/location.

L1 Cache

V	Tag	Data
		Line (64B)
...	...	...

L2 Cache

V	Tag	Data
		Line (128B)
...	...	...

DRAM (primary memory)

Page Table

Valid	PPN
1	
...	...
0	

Data Page

Disk (secondary memory)

Data Page (4KiB)

36-Virtual Memory II (32)



Caches vs. Virtual Memory

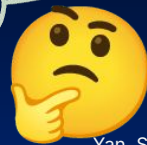
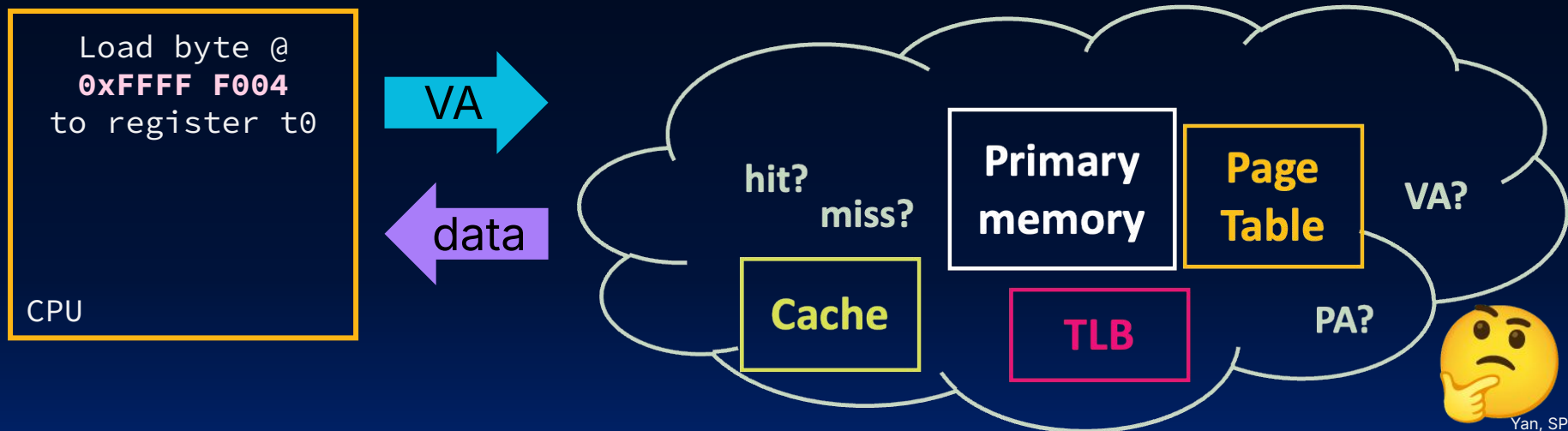
Caches

Virtual Memory

In memory hierarchy	Caches ↔ Memory	Memory ↔ Disk
Memory unit	Block (~64 bytes)	Page (~4096 bytes)
Miss	Cache Miss	Page Fault
Associativity	Direct-mapped, N-way set associative, fully associative	Fully associative (pages can go anywhere in memory)
Replacement policy	Least-recently-used (LRU) or random	LRU/approximation (most common), FIFO, or random
Write policy	Write-through or write-back	Write-back

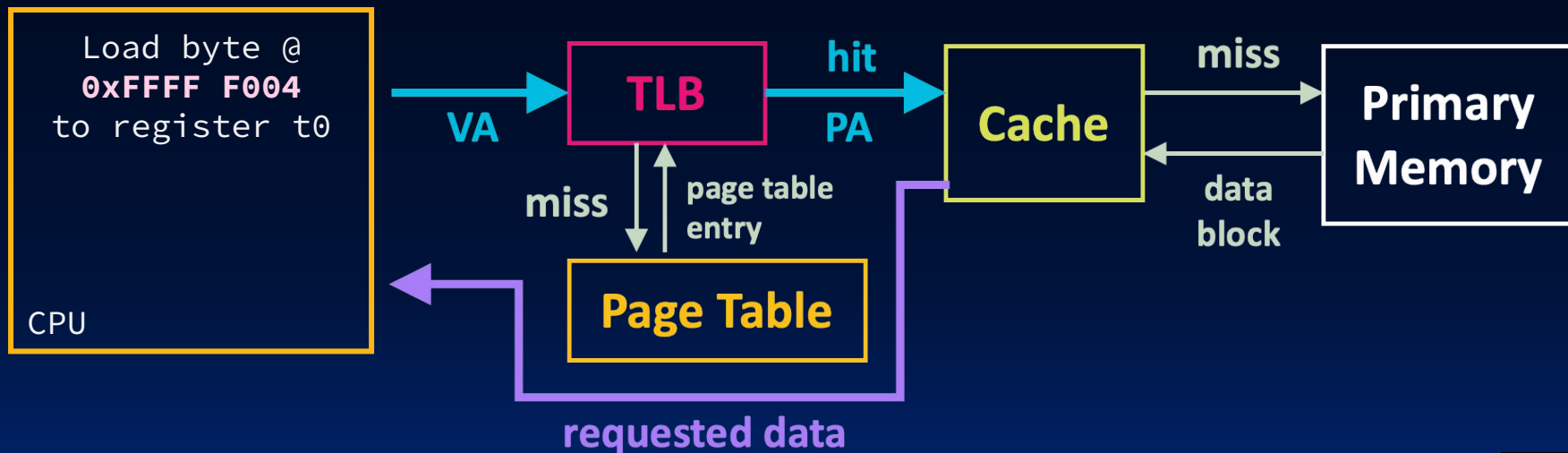
Putting It All Together: PIPT (1/2)

- Can a cache hold the requested data if the corresponding page is not in main memory?
- On a memory reference, what should we access first?
- When should we translate virtual addresses?



Putting It All Together: PIPT (2/2)

- Other more complicated designs exist, but in this class we assume **physically indexed, physically tagged (PIPT) caches**.
 - 1. **First** translate **virtual address** to physical address.
 - 2. Then use the **physical address** to access the data cache.





Reference Slides

- Page Table Details II
- The Translation Lookaside Buffer (TLB)
- Address Translation: Best/Worst Case
- Caches vs. Virtual Memory

1. Valid Bit: Demand Paging

- Page table entries track a **valid bit** to indicate if the page is in memory (DRAM) or only on disk.
 - On each memory access, first check if page table entry is "valid."
- Valid/on → In main memory
 - Read/write data as directed by process.
- Not Valid/off → On disk
 - Trigger **page fault** exception
 - OS intervenes to allocate the page into DRAM (more later)
 - If out of memory, first select a page to replace in DRAM
 - Store outgoing page to disk.
 - Read requested page from disk into DRAM.
 - Finally, read/write data as directed by process.



2. Dirty Bit: Memory's Write Policy

- Main memory acts like a "cache" for secondary memory/disk.
 - Should writes always go directly to disk (write-through), or
 - Should writes only go to disk when page is evicted (write-back)?
- All virtual memory systems use **write-back**.
 - Disk accesses take way too long!
- Recall that write-back policy requires a **dirty bit** to keep track of pages that have an unsynced write.
 - When a page gets replaced:
 - Dirty bit on: Write outgoing page back to disk.
 - Dirty bit off: No disk write.

3, aside. Program Isolation: Goals and Solutions

- OS goals for protection:
 - Isolate memory between processes.
 - Each process gets dedicated “private” memory.
 - Errors in one program won’t corrupt memory of other programs.
 - Prevent user programs from messing with OS’s memory.
 - Allow memory sharing where safe.
- Protection with Page Tables:
 1. One page table per process
 2. Memory sharing + **write protection bit** on page table entries